Velocity

The Kashmiri Programming Language

v0.2.0 • Reference & Guide

x86-64 ELF64 PE64 C99 compiler λ-calc GC NASM

compiled statically typed **structs** tuples optionals
try/catch panic **dual-platform** Linux & Windows
lambdas map filter **tiny C99 compiler**

Basit Ahmad Ganie
basitahmed1412@gmail.com

MIT License • Copyright © 2026

Preface

About This Book

This document is the official reference and programming guide for **Velocity**, a compiled, statically-typed systems programming language whose keywords are drawn from **Kashmiri** and **Urdu**. It targets native x86-64 machine code on both Linux (ELF64) and Windows (PE64), with no runtime or garbage collector.

Velocity sits in the same ecological niche as C — direct memory access, predictable performance, no hidden overhead — but adds structured error handling, first-class arrays, tuples, optional types, and a compilation pipeline that produces tight native binaries via NASM.

This guide is written in the spirit of Kernighan & Ritchie's *The C Programming Language* and *The Rust Book*: it builds knowledge systematically from first principles, uses real runnable code in every section, and trusts the reader to experiment at the terminal.

What you will find here:

- A conceptual overview of Velocity's design and compilation pipeline
- A complete keyword and type reference with Kashmiri etymology
- Coverage of all language features: variables, functions, control flow, arrays, structs, error handling, modules, and lambdas
- Annotated real-world examples drawn from the Velocity source tree
- Standard library documentation for all built-in modules
- Platform notes for Linux and Windows targets

How to read it: Chapters 1–3 give you enough to write and compile your first programs. Chapters 4–8 cover the full language feature set. Chapters 9–10 document the standard library and toolchain. The appendices collect quick-reference tables.

Basit Ahmad Ganie
Kashmir, 2026

Contents

Preface i

1 Introduction to Velocity 1
- 1.1 What Is Velocity? 1
- 1.2 The Compilation Pipeline 1
- 1.3 Building the Compiler 2
  - 1.3.1 Linux (native) 2
  - 1.3.2 Cross-Compiling for Windows from Linux 2
  - 1.3.3 Windows Native (MSYS2 / MinGW) 2
  - 1.3.4 Compiler CLI Reference 2

2 Your First Velocity Program 3
- 2.1 Hello, World! 3
  - 2.1.1 Compile and Run 3
- 2.2 Program Structure 4

3 Variables and Types 5
- 3.1 Declaring Variables 5
- 3.2 Primitive Types 6
- 3.3 Integer Literals 6
- 3.4 Compound Types 6
  - 3.4.1 Optional Types 6
- 3.5 Compound Assignment 7

4 Functions 8
- 4.1 Defining Functions 8
- 4.2 The Return Statement — `chu` 8
- 4.3 Recursion 9
- 4.4 Lambdas and Higher-Order Functions 9

5 Control Flow 10
- 5.1 Conditionals — `agar` / `nate` 10
- 5.2 Loops 10
  - 5.2.1 `yeli` — While Loop 10
  - 5.2.2 `bar manz` — Range For Loop 11
  - 5.2.3 `bar manz` — Iterating Arrays 11
  - 5.2.4 C-Style For Loop 11
  - 5.2.5 `phutraw` and `pakh` 12

6 Arrays, Tuples, and Strings 13
- 6.1 Fixed-Size Arrays 13
- 6.2 Dynamic Arrays 13
- 6.3 Arrays of Structs 14
- 6.4 Tuples 14
- 6.5 Strings — `lafz` 14

7 Structs **16**
7.1 Defining Structs . . . 16
7.2 Creating and Using Struct Values . . . 16
7.3 Nested Structs and Array Fields . . . 17
7.4 Field Access on Array Elements . . . 17

8 Bitwise and Logical Operations **18**
8.1 Bitwise Operators . . . 18
8.2 Logical Operators . . . 19

9 Error Handling **20**
9.1 Philosophy . . . 20
9.2 **panic** . . . 20
9.3 **trayith** and **koshish** / **ratt** . . . 20
9.4 The Discard Operator _ . . . 21

10 Modules and the Standard Library **22**
10.1 Importing Modules . . . 22
10.2 The io Module . . . 22
10.3 The lafz (String) Module . . . 23
10.4 The math Module . . . 23
10.5 The random Module . . . 24
10.6 The time Module . . . 24
10.7 The system and os Modules . . . 24

11 A Complete Example: Hashmap **25**

12 Platform Notes **27**
12.1 Dual-Platform Code Generation . . . 27
12.2 Calling Conventions . . . 27
12.3 Standard Library Architecture . . . 27
12.4 Windows Setup . . . 28

A Quick Reference — Keywords **29**

B Quick Reference — Types **30**

C Quick Reference — Format Specifiers **31**

D Quick Reference — Compiler Flags **32**

About the Author **33**

Chapter 1

Introduction to Velocity

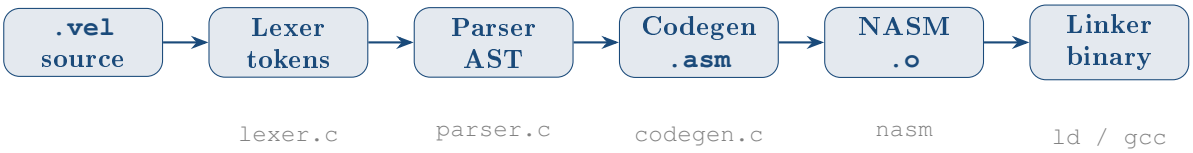
1.1 What Is Velocity?

Velocity is a **compiled, statically-typed, systems-level programming language**. It occupies the same niche as C — direct hardware access, deterministic execution, no garbage collector — but brings:

- **Kashmiri/Urdu keywords.** The language reads like Kashmiri prose. **`kar`** defines a function; **`ath`** declares a variable; **`agar`** introduces a condition; **`bar`** begins a loop.
- **Structured error handling.** **`koshish`**/**`ratt`** (try/catch) and **`panic`** bring Rust-flavoured error semantics in a simple, accessible form.
- **First-class arrays and tuples.** Fixed and dynamic arrays, structs with array fields, tuples, and optional types are all supported natively.
- **Dual-platform output.** One source file compiles to Linux ELF64 or Windows PE64 by passing a single flag.
- **A tiny, auditable compiler.** The entire compiler is plain C99 and fits in six source files (around 300 KB of combined source).

1.2 The Compilation Pipeline

Every Velocity source file follows one path from text to executable:



1. **Lexer** (`lexer.c`) — Tokenises the source; handles Kashmiri keywords, all operator forms, and multi-base integer literals (`0xFF`, `0b101`, `0o17`).
2. **Parser** (`parser.c`) — Builds a typed Abstract Syntax Tree from the token stream.

3. **Code Generator** (`codegen.c`) — Walks the AST and emits x86-64 NASM assembly; branches on target platform to select System V or Microsoft x64 calling conventions.
4. **NASM** — Assembles to a native object file.
5. **Linker** — Links the object with the standard library to produce an ELF64 or PE64 binary.

1.3 Building the Compiler

1.3.1 Linux (native)

```
cd velocity-0.2.0
make
./velocity --version
```

The `Makefile` compiles all C sources with `gcc -O2 -std=c99`.

1.3.2 Cross-Compiling for Windows from Linux

```
sudo apt install mingw-w64
make cross-win
# produces velocity.exe -- copy with stdlib/ to Windows
```

1.3.3 Windows Native (MSYS2 / MinGW)

```
gcc -O2 -std=c99 -o velocity.exe \
    main.c lexer.c parser.c codegen.c module.c
```

1.3.4 Compiler CLI Reference

```
velocity <source.vel> [options]

  -o, --output <file>   Output binary name (default: a.out)
  -v, --verbose         Keep .asm/.o; print NASM + ld commands
  -V, --version         Print version and exit
  -h, --help            Print help and exit
  --linux               Target Linux ELF64 (default on Linux)
  --windows             Target Windows PE64 (MS x64 ABI)
  --stdlib <path>       Override stdlib search path
  --emit-asm            Keep generated .asm file
  --no-link             Stop after assembly (produce .o only)
```

Environment Variable

Set `VELOCITY_STDLIB=/path/to/stdlib` to avoid passing `-stdlib` on every invocation.

Chapter 2

Your First Velocity Program

2.1 Hello, World!

Listing 2.1: hello.vel — the canonical first program

```
anaw io;

kar main() -> adad {
    io.chhaap("Hello, World!\n");
    chu 0;
}
```

Line by line:

`anaw io;`
: Import the `io` module. The keyword **`anaw`** means "bring" in Kashmiri. Without this import, printing functions are not in scope.

`kar main() -> adad {`
: Define a function named `main` returning a signed 64-bit integer (**`adad`**). Every Velocity program starts at `main`.

`io.chhaap("Hello, World!")`
: Call `chhaap` ("print" in Kashmiri) from the `io` module. The `\n` escape produces a newline.

`chu 0;`
: Return the value 0 to the operating system. **`chu`** is the Kashmiri return statement; 0 signals success by convention.

2.1.1 Compile and Run

```
./velocity hello.vel -o hello --stdlib ./stdlib
./hello
# Hello, World!
```

2.2 Program Structure

Every Velocity program follows this layout:

1. Module imports (**anaw**) at the top.
2. Struct definitions (**bina**).
3. Function definitions (**kar**).
4. Execution begins in `kar main() -> adad`.

Statements end with a semicolon `;`. Comments begin with # and run to the end of the line.

Chapter 3

Variables and Types

3.1 Declaring Variables

ath introduces a binding. Variables are **immutable by default**. Adding **mut** permits reassignment.

Listing 3.1: Variable declarations

```
anaw io;

kar main() -> adad {
    ath x = 42;                     # immutable; type inferred as adad
    ath mut y: ashari = 3.14;       # mutable 64-bit float
    ath name: lafz = "Velocity";    # string
    ath byte_val: uadad8 = 255;     # unsigned 8-bit integer

    # x = 99;     -- ERROR: x is immutable
    y = 2.71828; # OK: y is mutable

    io.chhaap("x=%d y=%f name=%s byte=%d\n", x, y, name, byte_val);
    chu 0;
}
```

Type Inference

When you write `ath x = 42;` without an explicit annotation, Velocity infers the type from the initialiser. Integer literals default to **adad**; floats to **ashari**; string literals to **lafz**; booleans to **bool**.

3.2 Primitive Types

Type	Description	Range / Notes
adad	Signed 64-bit integer	-2^{63} to $2^{63}-1$
uadad	Unsigned 64-bit integer	0 to $2^{64}-1$
uadad8	Unsigned 8-bit integer	0 to 255 (byte)
ashari	64-bit float (f64)	IEEE 754 double precision
ashari32	32-bit float (f32)	IEEE 754 single precision
bool	Boolean	`poz` / `apuz`
lafz	UTF-8 string	heap-allocated
khali	Void	return type only

3.3 Integer Literals

Velocity supports four bases for integer literals:

Listing 3.2: Integer literal formats

```
ath decimal = 255;          # base 10
ath hex     = 0xFF;         # base 16 -- prefix 0x
ath binary  = 0b11111111;   # base  2 -- prefix 0b
ath octal   = 0o377;        # base  8 -- prefix 0o
# All four hold the value 255
```

3.4 Compound Types

Type syntax	Meaning
`[T]`	Dynamic array of T, heap-allocated, growable
`[T; N]`	Fixed array of N elements of T, on the stack
`(T1, T2)`	Tuple with elements of types T1, T2, ...
`T?`	Optional T — either a value or `null`
`bina Name`	User-defined struct

3.4.1 Optional Types

Any type suffixed with `?` can hold either a concrete value or **null**:

Listing 3.3: Optional types

```
ath mut maybe: adad? = 42;
maybe = null;                  # now empty

agar maybe == null {
    io.chhaap("nothing here\n");
}
```

3.5 Compound Assignment

Listing 3.4: Compound assignment operators

```
ath mut score = 100;
score += 50;    # 150
score -= 25;    # 125
score *= 2;     # 250
score /= 5;     # 50
```

Chapter 4

Functions

4.1 Defining Functions

Functions are declared with **kar** ("to do/make") followed by the name, parameter list, return type, and body:

Listing 4.1: Function definitions

```
anaw io;

kar add(a: adad, b: adad) -> adad {
    chu a + b;
}

kar greet(name: lafz) -> khali {
    io.chhaap("Salaam, %s!\n", name);
}

kar main() -> adad {
    ath result = add(10, 32);
    io.chhaap("10 + 32 = %d\n", result);
    greet("Basit");
    chu 0;
}
```

Return type khali

Use **khali** ("empty") when a function produces no value — equivalent to `void` in C. A **khali** function may omit the **chu** statement.

4.2 The Return Statement — chu

chu evaluates its operand and immediately exits the enclosing function:

Listing 4.2: Early return

```
kar abs_val(n: adad) -> adad {
```

```
    agar n < 0 {
        chu 0 - n;     # early return
    }
    chu n;
}
```

4.3 Recursion

Listing 4.3: Recursive factorial

```
anaw io;

kar factorial(n: adad) -> adad {
    agar n <= 1 {
        chu 1;
    }
    chu n * factorial(n - 1);
}

kar main() -> adad {
    io.chhaap("10! = %d\n", factorial(10));
    chu 0;
}
```

4.4 Lambdas and Higher-Order Functions

Velocity supports lambda expressions for use with the built-in `map` and `filter` higher-order functions:

Listing 4.4: Lambda with map and filter

```
anaw io;

kar main() -> adad {
    ath a: [adad] = [1, 2, 3, 4];

    # Multiply every element by 10
    ath b = map(a, |x: adad| x * 10);

    # Keep only even elements
    ath c = filter(a, |x: adad| x % 2 == 0);

    io.chhaap("mapped:   %d %d %d %d\n", b[0], b[1], b[2], b[3]);
    io.chhaap("filtered: len=%d  %d %d\n", len(c), c[0], c[1]);
    chu 0;
}
```

The lambda syntax `|param: type| expression` defines an anonymous function whose single expression is returned implicitly.

Chapter 5

Control Flow

5.1 Conditionals — agar / nate

agar opens a conditional. **nate agar** adds else-if branches. **nate** provides the default branch:

Listing 5.1: Conditional chains

```
anaw io;

kar classify(n: adad) -> khali {
    agar n < 0 {
        io.chhaap("negative\n");
    } nate agar n == 0 {
        io.chhaap("zero\n");
    } nate agar n < 10 {
        io.chhaap("small positive\n");
    } nate agar n < 100 {
        io.chhaap("medium\n");
    } nate {
        io.chhaap("large\n");
    }
}

kar main() -> adad {
    classify(-5);
    classify(0);
    classify(7);
    classify(42);
    classify(999);
    chu 0;
}
```

5.2 Loops

5.2.1 yeli — While Loop

yeli ("when/while") repeats its body as long as the condition holds:

Listing 5.2: While loop countdown

```
ath mut x = 10;
yeli x > 0 {
    io.chhaap("%d ", x);
    x -= 1;
}
io.chhaap("\n");
# prints: 10 9 8 7 6 5 4 3 2 1
```

5.2.2 bar manz — Range For Loop

The most idiomatic loop in Velocity. Three range forms are supported:

Listing 5.3: Range loops

```
# Exclusive upper bound: 0, 1 ... 9
bar i manz 0..10 {
    io.chhaap("%d ", i);
}

# Inclusive upper bound: 1, 2 ... 5
bar i manz 1..=5 {
    io.chhaap("%d ", i);
}

# Stepped range: 0, 10, 20 ... 90
bar i manz 0..100 by 10 {
    io.chhaap("%d ", i);
}
```

Syntax	Meaning
`a..b`	$a \leq i < b$ (exclusive upper)
`a..=b`	$a \leq i \leq b$ (inclusive upper)
`a..b by s`	step by s, exclusive
`a..=b by s`	step by s, inclusive

5.2.3 bar manz — Iterating Arrays

Listing 5.4: Array iteration

```
ath nums: [adad] = [10, 20, 30, 40];
bar n manz nums {
    io.chhaap("%d\n", n);
}
```

5.2.4 C-Style For Loop

For situations requiring an explicit initialiser, test, and increment:

Listing 5.5: C-style for loop

```
bar (ath mut i = 0; i < 5; i += 1) {
    io.chhaap("i = %d\n", i);
}
```

5.2.5 phutraw and pakh

phutraw ("break") exits the innermost loop immediately. **pakh** ("continue") skips the rest of the current iteration:

Listing 5.6: Break and continue

```
bar i manz 0..20 {
    agar i % 2 == 0 {
        pakh;        # skip even numbers
    }
    agar i == 15 {
        phutraw;     # stop at 15
    }
    io.chhaap("%d\n", i);
}
# prints odd numbers: 1 3 5 7 9 11 13
```

Chapter 6

Arrays, Tuples, and Strings

6.1 Fixed-Size Arrays

A fixed-size array lives on the stack. Its length is a compile-time constant:

Listing 6.1: Fixed-size array

```
anaw io;

kar main() -> adad {
    ath mut scores: [adad; 5] = [100, 95, 88, 72, 60];
    scores[2] = 90;
    io.chhaap("len=%d  scores[2]=%d\n", len(scores), scores[2]);
    # len=5  scores[2]=90
    chu 0;
}
```

6.2 Dynamic Arrays

A dynamic array is heap-allocated and grows via `.append()`:

Listing 6.2: Dynamic array with append

```
anaw io;

kar main() -> adad {
    ath mut names: [lafz] = [];

    names.append("Shahid");
    names.append("Basit");
    names.append("Muneeb");

    io.chhaap("count: %d\n", len(names));

    bar n manz names {
        io.chhaap("  -> %s\n", n);
    }
    chu 0;
```

```
}
```

Velocity v0.2.0 does not insert automatic bounds checks. Accessing `arr[i]` when `i >= len(arr)` produces undefined behaviour. Always validate indices before indexing.

6.3 Arrays of Structs

Listing 6.3: Dynamic array of struct values

```
anaw io;

bina Point {
    x: ashari;
    y: ashari;
}

kar main() -> adad {
    ath mut pts: [Point] = [];
    pts.append(Point { x: 1.0, y: 2.0 });
    pts.append(Point { x: 3.5, y: -1.0 });

    bar p manz pts {
        io.chhaap("(%f, %f)\n", p.x, p.y);
    }
    chu 0;
}
```

6.4 Tuples

A tuple groups a fixed number of heterogeneous values. Elements are accessed with zero-based dot notation (`t.0`, `t.1`, ...):

Listing 6.4: Tuple literal and element access

```
anaw io;

kar main() -> adad {
    ath t = (42, 3.14, "hello");

    io.chhaap("t.0=%d  t.1=%f  t.2=%s\n", t.0, t.1, t.2);
    io.chhaap("len=%d\n", len(t));
    chu 0;
}
```

6.5 Strings — `lafz`

Strings are UTF-8 heap-allocated values. The `lafz` module provides utilities for string manipulation:

Listing 6.5: String operations

```
anaw io;
anaw lafz;

kar main() -> adad {
    ath s1: lafz = "Hello";
    ath s2: lafz = ", Kashmir";
    ath s3 = lafz.concat(s1, s2);

    io.chhaap("combined : %s\n", s3);
    io.chhaap("length   : %d\n", lafz.len(s3));

    ath sub = lafz.slice(s3, 0, 5);
    io.chhaap("first 5  : %s\n", sub);

    ath n = lafz.to_adad("42");
    io.chhaap("parsed   : %d\n", n);
    chu 0;
}
```

Chapter 7

Structs

7.1 Defining Structs

bina ("foundation") defines a named collection of typed fields:

Listing 7.1: Simple struct definition

```
bina Vec2 {
    x: ashari;
    y: ashari;
}
```

7.2 Creating and Using Struct Values

Listing 7.2: Struct instantiation and field access

```
anaw io;

bina Vec2 {
    x: ashari;
    y: ashari;
}

kar main() -> adad {
    ath mut pos = Vec2 { x: 3.0, y: 4.0 };

    io.chhaap("x=%f  y=%f\n", pos.x, pos.y);

    pos.x = 10.0;    # field assignment
    io.chhaap("after update: x=%f\n", pos.x);
    chu 0;
}
```

7.3 Nested Structs and Array Fields

Structs can contain other structs and dynamic array fields:

Listing 7.3: Nested structs with array field

```
anaw io;

bina Vec2   { x: ashari; y: ashari; }
bina Color  { r: uadad8; g: uadad8; b: uadad8; }

bina Particle {
    pos:  Vec2;       # nested struct
    vel:  Vec2;
    mass: ashari;
    tags: [lafz];     # dynamic array field
    id:   adad;
}

kar main() -> adad {
    ath mut p = Particle {
        pos:  Vec2  { x: 1.0, y: 2.0 },
        vel:  Vec2  { x: 0.5, y: -0.3 },
        mass: 1.5,
        tags: [],
        id:   42
    };

    p.pos.x = 10.0;           # chained field assignment
    p.tags.append("fast");

    io.chhaap("id=%d  x=%f  tags=%d\n",
              p.id, p.pos.x, len(p.tags));
    chu 0;
}
```

Constructor Functions

Velocity allows you to define a function with the same name as a struct to act as a constructor, returning a fully initialised struct value. This is a convention, not a language-enforced feature.

7.4 Field Access on Array Elements

Listing 7.4: Struct field access inside a loop

```
bar i manz table {
    io.chhaap("key=%d  value=%s\n", i.key, i.value);
}
```

Chapter 8

Bitwise and Logical Operations

8.1 Bitwise Operators

Op	Name	Example (decimal)
&	AND	10 & 12 → 8
|	OR	10 | 12 → 14
^	XOR	10 ^ 12 → 6
~	NOT	~10 → all bits flipped
«	Left shift	1 « 4 → 16
»	Right shift	256 » 3 → 32

Listing 8.1: Bitwise operations

```
anaw io;

kar main() -> adad {
    ath a = 0b1010;     # 10
    ath b = 0b1100;     # 12

    io.chhaap("AND  : %d\n", a & b);     # 8
    io.chhaap("OR   : %d\n", a | b);     # 14
    io.chhaap("XOR  : %d\n", a ^ b);     # 6
    io.chhaap("SHL 2: %d\n", a << 2);    # 40
    io.chhaap("SHR 1: %d\n", a >> 1);    # 5

    # Test if bit 1 is set
    ath flags = 0b1010;
    agar flags & 0b0010 != 0 {
        io.chhaap("bit 1 is set\n");
    }
    chu 0;
}
```

8.2 Logical Operators

Boolean logic uses either C-style symbols or Kashmiri words:

Symbol	Kashmiri	Meaning
`&&`	**wa**	Logical AND
`||`	**ya**	Logical OR
`!`	**na**	Logical NOT

Listing 8.2: Logical operators — both forms

```
ath a = poz;        # true
ath b = apuz;       # false

# C-style symbols
ath c = a && b;     # false
ath d = a || b;     # true
ath e = !a;         # false

# Kashmiri words (identical semantics)
ath f = a wa b;     # false
ath g = a ya b;     # true
ath h = na a;       # false
```

The Kashmiri words **wa**, **ya**, **na** read naturally in complex boolean conditions: `agar x > 0 wa x < 100` reads almost like natural Kashmiri — "if x is greater than zero *and* x is less than one hundred".

Chapter 9

Error Handling

9.1 Philosophy

Velocity provides three complementary error-handling mechanisms:

1. **panic(msg)** — for unrecoverable errors (like Rust's `panic!`).
2. **trayith expr** — throw an error value up the call stack.
3. **koshish** / **ratt** — a try/catch boundary to intercept thrown errors.

9.2 panic

panic prints a message to stderr and terminates the program immediately. Use it for programming errors: violated preconditions, unreachable states, etc.

Listing 9.1: panic for unrecoverable errors

```
kar safe_divide(a: adad, b: adad) -> adad {
    agar b == 0 {
        panic("safe_divide: division by zero!");
    }
    chu a / b;
}
```

9.3 trayith and koshish / ratt

trayith propagates a **lafz** error value through the call stack. **koshish**/**ratt** defines a recovery boundary:

Listing 9.2: try / throw / catch

```
anaw io;

kar risky(x: adad) -> adad {
    agar x < 0 {
        trayith "negative input not allowed";
```

```
    }
    chu x * 2;
}

kar main() -> adad {
    # Simple throw
    koshish {
        trayith "boom";
    } ratt (e) {
        io.chhaap("caught: %s\n", e);
    }

    # Calling a function that might throw
    koshish {
        ath r = risky(-5);
        io.chhaap("result: %d\n", r);
    } ratt (e) {
        io.chhaap("recovered: %s\n", e);
    }

    io.chhaap("execution continues\n");
    chu 0;
}
```

Error Values

In v0.2.0, thrown error values are always **lafz** (string). The variable bound in the **ratt (e)** clause holds this string for inspection or display.

9.4 The Discard Operator _

When a function returns a value you deliberately do not need, assign it to _ to make the intent explicit:

```
_ = expensive_function();
_ = "side-effect only expression";
```

Chapter 10

Modules and the Standard Library

10.1 Importing Modules

anaw ("bring/fetch") imports a module. All stdlib modules have single-word names:

```
anaw io;
anaw lafz;
anaw math;
anaw random;
anaw time;
anaw system;
anaw os;
```

After importing, call functions with `module.function()` syntax.

10.2 The io Module

Function	Description
`io.chhaap(fmt, ...)`	Printf-style formatted print. Specifiers: `%d` int, `%f` float, `%s` string, `%b` bool, `%c` char, `%%` literal percent.
`io.chhaapLine()`	Print a newline character
`io.chhaapSpace()`	Print a space character
`io.stdin()`	Read a line from stdin; returns **lafz**
`io.input()`	Read an integer from stdin; returns **adad**

Listing 10.1: Interactive input

```
anaw io;

kar main() -> adad {
    io.chhaap("Enter your name: ");
    ath name = io.stdin();
    io.chhaap("Salaam, %s!\n", name);

    io.chhaap("Enter a number: ");
```

```
    ath n = io.input();
    io.chhaap("Double = %d\n", n * 2);
    chu 0;
}
```

10.3 The lafz (String) Module

Function	Description
`lafz.len(s)`	Number of bytes in the string
`lafz.eq(a, b)`	String equality; returns **bool**
`lafz.concat(a, b)`	Concatenate two strings
`lafz.slice(s, i, n)`	Substring: n bytes starting at index i
`lafz.from_adad(n)`	Convert **adad** to **lafz**
`lafz.from_ashari(f)`	Convert **ashari** to **lafz**
`lafz.to_adad(s)`	Parse **lafz** to **adad**
`lafz.to_ashari(s)`	Parse **lafz** to **ashari**

10.4 The math Module

Implemented in Velocity itself (`stdlib/math.vel`), using pure-Velocity arithmetic with no external library dependency.

Function	Signature	Notes
`math.sqrt(x)`	ashari → ashari	Newton-Raphson, 50 iters
`math.abs(x)`	ashari → ashari	Absolute value
`math.floor(x)`	ashari → ashari	Round toward $-\infty$
`math.ceil(x)`	ashari → ashari	Round toward $+\infty$
`math.pow(b, e)`	(ashari, adad) → ashari	Integer exponent
`math.min(a, b)`	adad, adad → adad	Minimum
`math.max(a, b)`	adad, adad → adad	Maximum
`math.clamp(x,l,h)`	adad, adad, adad → adad	Clamp to [l, h]

Listing 10.2: Using the math module

```
anaw io;
anaw math;

kar main() -> adad {
    ath root = math.sqrt(2.0);
    io.chhaap("sqrt(2)       = %f\n", root);

    ath p = math.pow(2.0, 10);
    io.chhaap("2^10          = %f\n", p);

    ath c = math.clamp(150, 0, 100);
    io.chhaap("clamp(150,0,100) = %d\n", c);
    chu 0;
}
```

10.5 The random Module

Uses an xorshift64* PRNG implemented in pure x86-64 assembly.

Function	Description
`random.seed(n)`	Seed the PRNG
`random.randrange(max)`	Uniform integer in $[0, \texttt{max})$
`random.random()`	Uniform float in $[0.0, 1.0)$

10.6 The time Module

Function	Description
`time.now()`	Seconds since Unix epoch (**adad**)
`time.now_ms()`	Milliseconds since epoch
`time.now_ns()`	Nanoseconds since epoch
`time.sleep(ms)`	Sleep for `ms` milliseconds
`time.ctime()`	Human-readable time string (**lafz**)

10.7 The system and os Modules

Function	Description
`system.argc()`	Number of command-line arguments
`system.argv(i)`	The i-th argument as **lafz**
`system.exit(n)`	Terminate with exit code `n`
`system.getenv(k)`	Value of environment variable `k`
`os.getcwd()`	Current working directory as **lafz**
`os.chdir(path)`	Change working directory

Chapter 11

A Complete Example: Hashmap

This chapter walks through a non-trivial Velocity program — a simple lookup table built on top of dynamic struct arrays. It demonstrates how language features compose in a real-world scenario.

Listing 11.1: hashmap.vel — full listing

```
# A hashmap-like lookup table using struct arrays.
# Demonstrates: structs, dynamic arrays, append,
#               for-in iteration, and panic.

anaw io;

bina Entry {
    key:   adad;
    value: lafz;
    used:  bool;
}

# Constructor function (same name as struct)
kar Entry(k: adad, v: lafz, u: bool) -> Entry {
    chu Entry { key: k, value: v, used: u };
}

# Alternative helper constructor
kar make_table_entry(k: adad, v: lafz) -> Entry {
    chu Entry { key: k, value: v, used: poz };
}

# Linear search: panic if key not found
kar get(table: [Entry], k: adad) -> lafz {
    bar t manz table {
        agar t.key == k {
            chu t.value;
        }
    }
    panic("no such key found!\n");
}

kar hashmap_demo() -> adad {
    ath mut table: [Entry] = [];
```

```

    # Insert entries via constructor
    table.append(Entry(4, "Aiyaan",  poz));
    table.append(Entry(5, "Musaib",  poz));
    table.append(Entry(6, "Kifayet", poz));

    # Insert via helper
    table.append(make_table_entry(1, "Shahid"));
    table.append(make_table_entry(2, "Basit"));
    table.append(make_table_entry(3, "Muneeb"));

    io.chhaap("Table size: %d\n", len(table));

    bar i manz table {
        io.chhaap("  key: %d  value: %s\n", i.key, i.value);
    }

    io.chhaap("Enter a key: ");
    ath num: adad = io.input();
    io.chhaap("Key %d => %s\n", num, get(table, num));
    chu 0;
}

kar main() -> adad {
    hashmap_demo();
    chu 0;
}
```

Key patterns illustrated:

- Defining a **bina** struct with mixed field types.
- A constructor function sharing the struct name.
- Building a dynamic array (**[Entry]**) with `.append()`.
- Iterating with **bar t manz table**.
- Linear search using early **chu** inside a loop.
- Using **panic** as a "not found" sentinel.

Chapter 12

Platform Notes

12.1 Dual-Platform Code Generation

```
# Linux ELF64 (default on Linux host)
./velocity hello.vel -o hello --stdlib ./stdlib

# Windows PE64 (cross-compiled from Linux)
./velocity hello.vel -o hello.exe --windows --stdlib ./stdlib
```

12.2 Calling Conventions

The code generator switches ABI automatically:

Target	ABI	Argument Registers
Linux ELF64	System V AMD64	`rdi rsi rdx rcx r8 r9`
Windows PE64	Microsoft x64	`rcx rdx r8 r9` + 32-byte shadow

12.3 Standard Library Architecture

`io.asm`, `lafz.asm`, `random.asm`, `system.asm`, `os.asm`, `time.asm`
: NASM assembly files. Each uses `%ifdef WIN64` / `%ifdef LINUX` to select between Win32 API calls and raw Linux syscalls.

`math.vel`, `util.vel`
: Written in Velocity itself — pure arithmetic, no syscalls required. Compiled to native code along with user programs.

The dual-mode architecture means the same standard library source compiles correctly for both platforms without any user intervention.

12.4 Windows Setup

On Windows, install NASM and GCC (via MSYS2) and add both to `PATH`:

```
# In MSYS2 MinGW terminal:
pacman -S mingw-w64-x86_64-gcc

# Verify all tools are present:
nasm --version
gcc --version
velocity.exe --version

# Compile and run:
velocity.exe hello.vel -o hello.exe --stdlib C:\velocity\stdlib
hello.exe
```

Appendix A

Quick Reference — Keywords

Keyword	English Equivalent	Category
`kar`	fn / function	Declaration
`chu`	return	Declaration
`ath`	let	Declaration
`mut`	mutable	Modifier
`bina`	struct	Declaration
`anaw`	import	Module
`agar`	if	Control flow
`nate`	else	Control flow
`yeli`	while	Control flow
`bar`	for	Control flow
`manz`	in	Control flow
`phutraw`	break	Control flow
`pakh`	continue	Control flow
`koshish`	try	Error handling
`ratt`	catch	Error handling
`trayith`	throw	Error handling
`panic`	panic	Error handling
`poz`	true	Literal
`apuz`	false	Literal
`null`	null	Literal
`wa`	&&	Logical operator
`ya`	||	Logical operator
`na`	!	Logical operator

Appendix B

Quick Reference — Types

Type	Description	Width
`adad`	Signed integer	64 bits
`uadad`	Unsigned integer	64 bits
`uadad8`	Unsigned byte	8 bits
`ashari`	Float	64 bits (f64)
`ashari32`	Float	32 bits (f32)
`bool`	Boolean	poz / apuz
`lafz`	String (UTF-8)	heap pointer
`khali`	Void	return type only
`[T]`	Dynamic array	heap-allocated
`[T; N]`	Fixed array	stack-allocated
`(T1, T2)`	Tuple	stack-allocated
`T?`	Optional T	T or null

Appendix C

Quick Reference — Format Specifiers

Spec	Type	Example
`%d`	adad, uadad, uadad8	`"val=%d"`
`%f`	ashari, ashari32	`"pi=%f"`
`%s`	lafz (string)	`"name=%s"`
`%b`	bool	prints `true` or `false`
`%c`	character (adad)	prints the ASCII character
`%%`	literal percent	prints `%`

Appendix D

Quick Reference — Compiler Flags

Flag	**Effect**
`-o, -output` *`file`*	Set output binary name
`-v, -verbose`	Keep .asm/.o; print NASM and linker commands
`-V, -version`	Print version string and exit
`-h, -help`	Print usage and exit
`-linux`	Target Linux ELF64 (default on Linux host)
`-windows`	Target Windows PE64 (MS x64 ABI)
`-stdlib` *`path`*	Override stdlib search path
`-emit-asm`	Keep generated .asm file
`-no-link`	Stop after assembly; produce .o only

About the Author

Basit Ahmad Ganie is the creator of the Velocity programming language. He designed Velocity to demonstrate that a systems programming language does not need to be rooted in English — that the words of Kashmir carry the precision required to express computation.

Contact: basitahmed1412@gmail.com

MIT License — Copyright © 2026 Basit Ahmad Ganie.